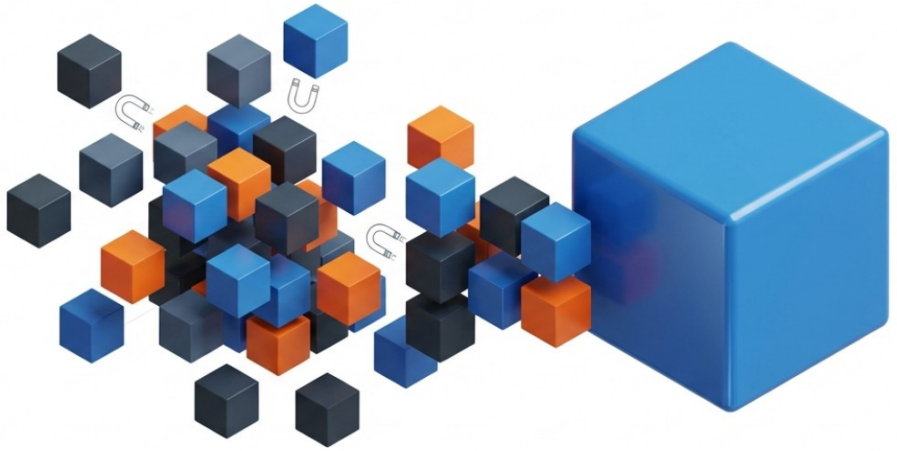
CHAPTER 5

ARRAYS AND STRINGS

C++ Programming • Introductory Course

Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering
Foundations of Programming



Store, traverse, and transform collections of values and text.

01

ARRAYS

A fixed-size sequence turns repeated variables into structured data.

Learning outcomes

By the end of this chapter, you can…

1. **Declare and initialize arrays**

   Choose an element type, fixed bound, and initial values.

2. **Traverse without crossing bounds**

   Use indices or range-based loops to process every element.

3. **Process matrices and text**

   Apply nested loops and character-by-character traversal.

4. **Choose a safer abstraction**

   Use std::array, std::span, or std::string when they express intent better.

Focus on readable, bounds-aware code—not memorizing syntax in isolation.

A four-part learning path

The sequence builds one mental model, then reuses it for text.

Each part answers a practical programming question.



Represent

How is data stored?

Traverse

How do we visit each value?

Extend

How do rows and columns work?

Transform

How is text processed?

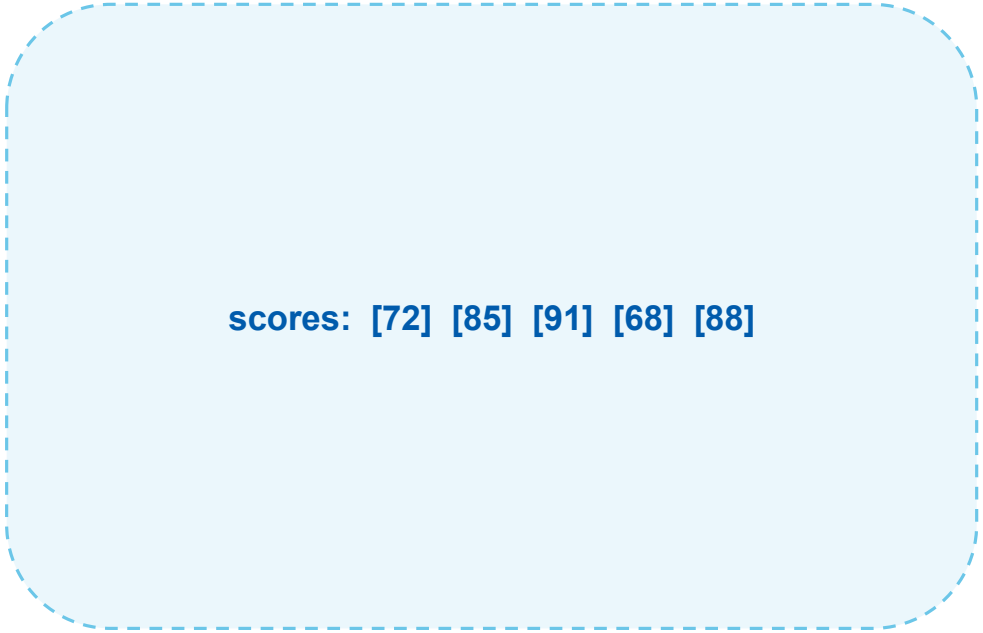
One sequence model connects numbers, matrices, and strings.

An array groups values under one name

Use an array when the element type and count are fixed.

An array stores N same-type elements in contiguous positions numbered 0 through N − 1.

The array name identifies the sequence; an index selects one element. Every index must stay inside the declared bound.



Five int elements; scores[0] is the first.

01 **Fixed size**

The bound is part of the type.

02 **Zero-based**

The first index is 0.

03 **One element type**

Every slot stores the same type.

Declare the type, name, and fixed bound

Initialization establishes useful values before the first read.

Declaration

- int scores[5];
- Element type: int
- Fixed bound: 5
- Valid indices: 0–4

Initialization

- int scores[5] = {72, 85, 91, 68, 88};
- int zeros[5] = {};
- int a[] = {4, 7, 9};
- Use braces to make intent visible.

An index selects exactly one element

Read scores[i] as “the value at position i.”

Index	Expression	Value	Meaning
0	scores[0]	72	first element
1	scores[1]	85	second element
2	scores[2]	91	third element
3	scores[3]	68	fourth element

INDEX

A position counted from 0

ELEMENT

The selected array slot

VALUE

Data stored in that slot

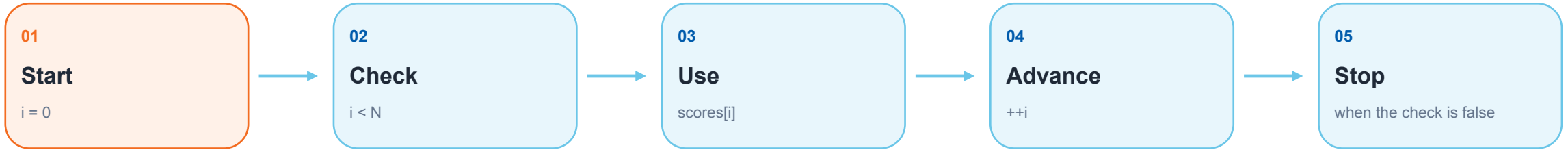
BOUND

For five elements: $0 \le i < 5$

The last valid index is size − 1; scores[5] is outside this five-element array.

A safe loop keeps the index inside the bound

Every access happens only after the loop condition succeeds.



Loop invariant: every array access occurs only while 0 ≤ i < N.

Example: compute the mean of five scores

Separate accumulation from the final division.

Algorithm

- Set sum to zero.
- Visit every value.
- Add each value to sum.
- Divide by 5.0 after the loop.

C++ implementation

- int sum = 0;
- for (int x : scores) sum += x;
- double mean = sum / 5.0;
- std::cout << mean;

Most array bugs are boundary mistakes

A single comparison operator can decide whether every access is valid.

WRONG: ONE STEP TOO FAR



Problem pattern

- for (int i = 0; i <= N; ++i)
- scores[N] is outside the array.
- Reading an uninitialized element is unsafe.

CORRECT: STOP BEFORE N



Bounds-aware pattern

- Use i < N for indexed loops.
- A range-based loop avoids the index.
- Check empty or special cases first.

Use < N, not <= N; prefer iteration patterns that make the bound explicit.

02

MODERN FIXED-SIZE INTERFACES

Make size and ownership visible in the type or function interface.

std::array adds a container interface to fixed-size data

Both choices store a fixed number of contiguous elements.

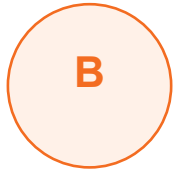
BUILT-IN ARRAY: T[N]



Language-level array

- Fixed-size storage with bracket syntax.
- Array parameters adjust to pointers.
- No member function reports the size.

std::array<T, N>



Standard-library container

- Carries N in its type; exposes .size().
- Supports iterators, .at(), fill, and swap.
- Works naturally with generic algorithms.

For new value-like fixed-size data, std::array is usually the clearer interface.

Functions need both the sequence and its size

An array parameter is adjusted to a pointer; the bound is not carried with it.

Classic interface

- double mean(const int data[], int n);
- data points to the first element.
- n tells the function how far to read.
- Caller and callee must agree on n.

C++20 size-aware view

- double mean(std::span<const int> data);
- data.size() travels with the view.
- The view does not own the elements.
- Use const when the function only reads.

A 2D array models rows and columns

int grid[3][3] stores nine ints; access uses grid[row][column].

Row	Column 0	Column 1	Column 2
0	1	2	3
1	4	5	6
2	7	8	9
Column sum	12	15	18

TYPE

int[3][3]

ROW

grid[r] selects one row

COLUMN

grid[r][c] selects one int

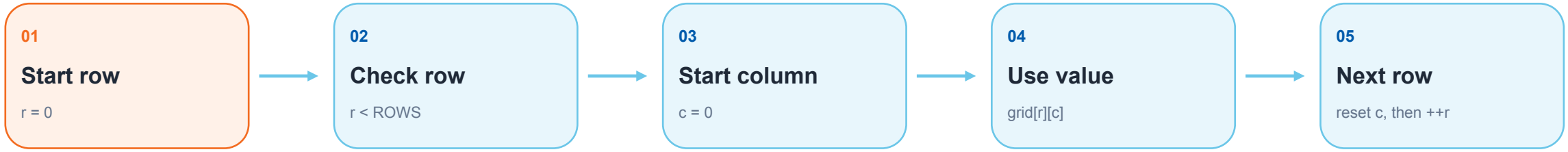
ORDER

Row index, then column index

Think grid[row][column]; each row is itself an array of three ints.

Nested loops visit every matrix element

The outer loop chooses a row; the inner loop walks across its columns.



For row-by-row traversal, keep the row loop outside and the column loop inside.

03

STRINGS

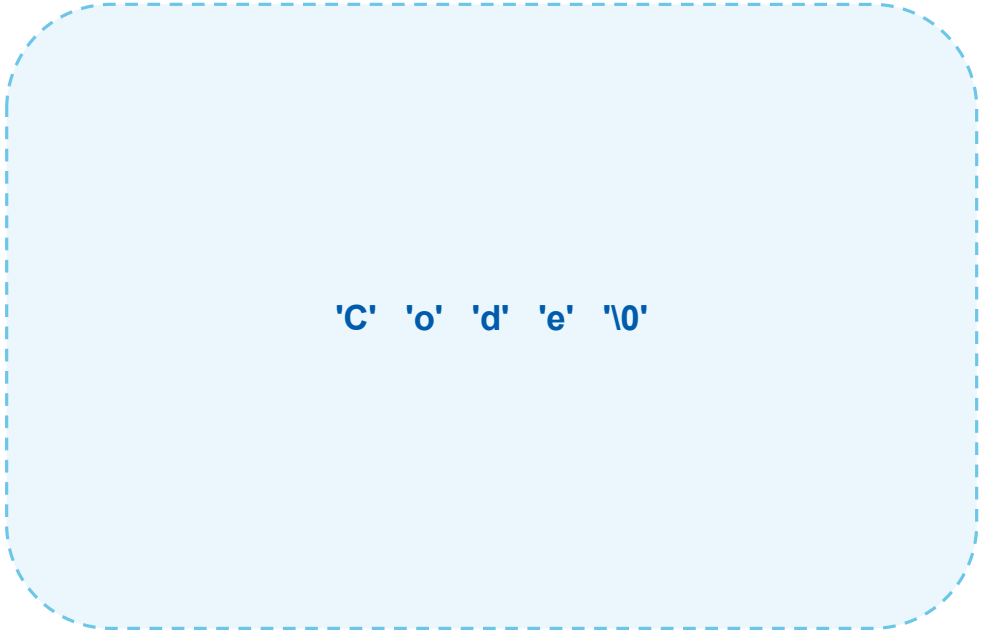
Text is a sequence of characters—but the representation changes the interface.

A C-style string is a char array ending with \0

The terminator marks the logical end of the text.

The null character marks where C-string operations must stop scanning.

char word[] = "Code"; creates five elements: four letters plus the trailing terminator. A missing terminator breaks C-string assumptions.



The terminator occupies storage but is not printed.

01 **Character array**

Each element has type char.

02 **Null-terminated**

Scanning stops at '\0'.

03 **Fixed capacity**

The buffer limits text length.

std::string manages text as a resizable sequence

Use the standard string interface for everyday C++ text.

Core operations

- std::string name = "Ada";
- name.size(), name.empty()
- name += " Lovelace";
- name[0] or name.at(0)

Search and edit

- name.find("Love")
- name.substr(0, 3)
- name.insert(...), name.erase(...)
- Prefer the library to manual buffers.

getline reads spaces; operator>> stops at whitespace

Choose the input operation that matches the data unit you need.

Token input

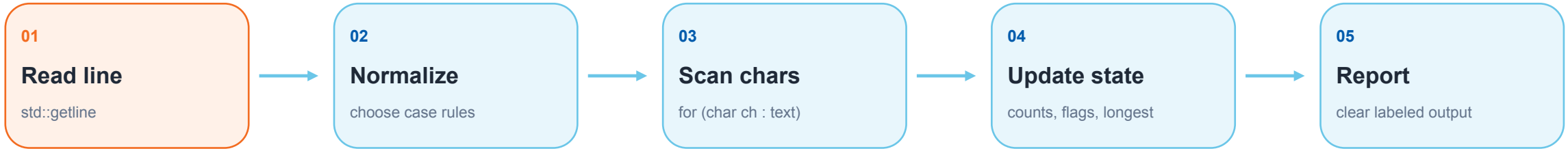
- std::cin >> name;
- Input: Ada Lovelace
- name becomes "Ada"
- Use for one whitespace-delimited token.

Line input

- std::getline(std::cin, name);
- Input: Ada Lovelace
- name becomes "Ada Lovelace"
- Mixing >> and getline? Handle the pending newline.

Text processing is sequence traversal plus character rules

Keep input storage, traversal, and counting state conceptually separate.



A clean solution stores the text once and derives results during a controlled traversal.

Choose the representation that matches the job

Both store characters; their size and safety interfaces differ.

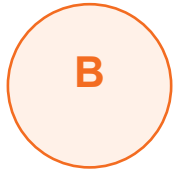
C-STYLE char[]



Manual buffer model

- Fixed capacity chosen by the programmer.
- Requires correct null termination.
- Useful for C APIs and low-level interop.

std::string



Library string model

- Resizable owning sequence of characters.
- Provides size, search, and edit operations.
- Default choice for everyday C++ text.

Prefer std::string unless a fixed buffer or C interface is an explicit requirement.

Practice: analyze a line of text

Combine strings, loops, decisions, and test cases.

Read one full line. Report its character count, spaces, vowels, and longest word.

Keep the original text in std::string. Traverse once for character counts; scan word boundaries to track the longest word.

Input: Arrays make data easier to process.

Expected output should clearly label every count.

01 **Read**

Use getline for the full line.

02 **Traverse**

Update counters and word state.

03 **Test**

Include empty and spaced input.

Check your bounds intuition

Choose the single best answer.

int a[4] = {10, 20, 30, 40}; Which expression accesses the last element?

A. a[4]

B. a[2]

C. a[3]

D. a[-1]

Answer: C. Four elements use indices 0, 1, 2, and 3.

Four habits make sequence code reliable

Representation, bounds, abstraction, and interfaces work together.

- **Representation:** Choose an element type and a size that match the data.
- **Bounds:** Access only indices 0 through size − 1.
- **Abstraction:** Use std::array for fixed-size values and std::string for text.
- **Interfaces:** Pass size information explicitly; consider std::span in C++20.

A clear sequence model turns loops, matrices, and strings into the same solvable pattern.

Further study

Standards-aligned references and optional extensions

Course extension

- Practice: frequency tables, palindromes, matrix sums
- Next: std::vector for run-time sizes

C++ working draft

- Arrays: eel.is/c++draft/dcl.array and /array.overview
- Strings: eel.is/c++draft/basic.string

Modern interfaces

- std::span: eel.is/c++draft/span.overview
- Function parameters: eel.is/c++draft/dcl.fct

CHAPTER 5

Key choices
Fixed size → std::array
Text → std::string
Run-time size → std::vector